

Comparative Analysis of Graph Traversal and Shortest Path Algorithms

Sumit Desai

Department of Computer Science & Engineering
COEP Technological University
Pune, India
MIS: 612415049

February 12, 2026

COEP Technological University [COEP Tech]
(A Unitary Public University of Government of Maharashtra)



1 Abstract

2 Introduction

3 Methodology

4 Results

5 Conclusion



6 References

COEP Technological University [COEP Tech]
(A Unitary Public University of Government of Maharashtra)

Abstract

- ◀ **Context:** Models pairwise relationships in complex systems (e.g., social networks, transportation).
- ◀ **Mathematical Definition:** Systems are modeled as a graph G defined in Eq. (1).

$$G = (V, E) \quad (1)$$

Where V represents vertices and E represents edges.

Algorithms Analyzed:

- 1 Breadth-First Search (BFS)
- 2 Depth-First Search (DFS)
- 3 Dijkstra's Algorithm



Introduction

Background and Motivation

- ◀ Graphs are widely used to model real-world systems such as networks, maps, and social platforms.
- ◀ Efficient traversal of graphs is essential for solving connectivity and routing problems.
- ◀ Early approaches focused on simple traversal without cost considerations.
- ◀ Traditional methods often ignored edge weights or scalability issues.
- ◀ Growing data sizes demand more efficient and optimal algorithms.
- ◀ This motivates the study of classical and shortest-path algorithms.

Problem Overview

Graph traversal and shortest path problems form the foundation of many computer science applications. While simple traversal techniques are sufficient for unweighted graphs, they fail to account for real-world constraints such as distance, cost, or time. This creates a need to analyze and compare different algorithms based on their behavior, efficiency, and suitability for specific graph types. Understanding these differences helps in selecting the right algorithm for practical applications.



Methodology: Breadth-First Search (BFS)

- ◀ Earlier traversal methods focused only on visiting nodes, not path optimality.
- ◀ BFS improves this by exploring graphs level by level.
- ◀ It guarantees the shortest path only when all edges have equal cost.
- ◀ BFS fails in weighted graphs because it ignores edge weights.
- ◀ This limitation demands more cost-aware approaches → Dijkstra.

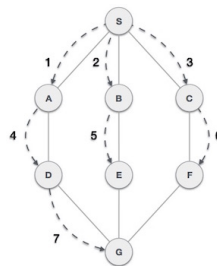


Figure: *

Level-wise expansion from source node



Methodology: Depth-First Search (DFS)

- ▶ DFS explores deeply before considering alternative paths.
- ▶ Earlier approaches using DFS were efficient for structure analysis.
- ▶ DFS does not guarantee shortest or optimal paths.
- ▶ It may traverse long paths even when shorter paths exist.
- ▶ Hence, DFS is unsuitable for routing or cost-based problems.

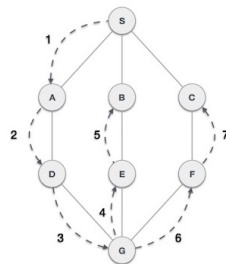


Figure: *

Deep traversal along a single branch

Methodology: Dijkstra's Algorithm

- ▶ BFS and DFS fail to handle weighted graphs correctly.
- ▶ Dijkstra introduces a greedy, cost-based selection strategy.
- ▶ It always expands the node with minimum cumulative cost.
- ▶ Uses a priority queue to ensure optimal path selection.
- ▶ Overcomes limitations of earlier traversal-based approaches.

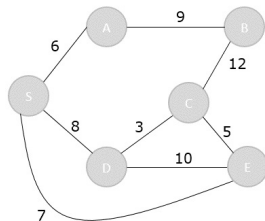


Figure: *

Shortest path based on cumulative edge weight



Results

- ◀ The comparative study shows that no single graph algorithm is optimal for all scenarios.
- ◀ BFS performs best for shortest path problems in unweighted graphs with minimal overhead.
- ◀ DFS is effective for structural analysis but unsuitable for optimal path discovery.
- ◀ Dijkstra's algorithm produces correct shortest paths in weighted graphs at a higher computational cost.
- ◀ Future work includes extending this study to algorithms handling negative weights and large-scale graphs.



Algorithm Comparison Summary

Metric	BFS	DFS	Dijkstra
Graph Type	Unweighted	Any	Weighted
Shortest Path	Yes	No	Yes
Time Complexity	$O(V + E)$	$O(V + E)$	$O(E \log V)$
Auxiliary Space	$O(V)$	$O(V)$	$O(V)$
Primary Use	Traversal	Structure	Routing



Conclusion

- ◀ A detailed literature review of classical graph algorithms was conducted.
- ◀ BFS, DFS, and Dijkstra's algorithm were analyzed theoretically and visually.
- ◀ Algorithm behavior was studied using traversal diagrams and complexity analysis.
- ◀ Comparative results highlight strengths and limitations of each approach.
- ◀ Future work includes implementing simulations and analyzing advanced shortest path algorithms.



References

- 1 T. H. Cormen et al., *Introduction to Algorithms*, MIT Press.
- 2 E. W. Dijkstra, "A note on two problems in connexion with graphs," 1959.
- 3 R. Sedgewick and K. Wayne, *Algorithms*, Addison-Wesley.
- 4 TutorialsPoint, "Graph traversal algorithms," Online resource.

